

A CAPSTONE PROJECT REPORT

TITLE: AlgoTrade Pro: A Production-Grade Algorithmic Trading Platform for Indian Stock Markets

Submitted by: Sunasara Haidarali

ROLL NO: 2411022250090

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

in

Data Science (Department of Computer Science & Engineering)

ALLIANCE UNIVERSITY, BANGALORE

MARCH 2026

STUDENT DECLARATION

I, Sunasara Haidarali, a student of MCA in Data Science under the Department of Computer Science & Engineering at Alliance University, bearing Registration No. 2411022250090, do hereby declare that:

1. The capstone project report titled "AlgoTrade Pro: A Production-Grade Algorithmic Trading Platform for Indian Stock Markets" is a record of the original work carried out by me under the guidance of Ms. Nagasri (Internal Guide) and Dr. J. Lenin (External Guide) during the period from 01 January 2026 to 15 April 2026.
2. The information, data, and analysis presented in this report are authentic and based on my actual project work, implementation, testing, and validation.
3. I have not copied or plagiarized any part of this report from any other project, thesis, or published material. Due credit has been given to all sources of information in the references section.
4. I have maintained confidentiality and avoided disclosure of sensitive credentials, private keys, and restricted project artifacts.
5. This work has not been submitted previously to any other University or Institution for the award of any degree or diploma.

Date: 27 April 2026

Place: Bangalore

Signature and Name of the Student

EXECUTIVE SUMMARY

This capstone project presents the design and implementation of **AlgoTrade Pro**, a full-stack algorithmic trading platform focused on Indian equity markets (NSE/BSE). The objective is to transform raw market data into actionable trading decisions through an integrated system that combines

real-time broker connectivity, technical analytics, risk controls, and AI-assisted intelligence.

The platform is built with **FastAPI (Python)** on the backend and **React with TypeScript (Vite)** on the frontend. The backend uses **PostgreSQL** with asynchronous access through **SQLAlchemy 2.0 and asyncpg**, while real-time price updates are delivered over WebSockets. The architecture follows a clean service-oriented approach: routers handle APIs, services handle business logic, models define persistence, and configuration is centralized through a Pydantic settings layer.

Major implemented modules include paper trading with persistent trade history, backtesting with realistic Indian cost assumptions, technical analysis using multiple indicators, broker integration (Angel One and Zerodha), and an AI analysis layer powered by Google Gemini with graceful fallback behavior under rate-limit conditions. The project also implements security safeguards such as JWT authentication, encrypted credential vaulting, and pre-trade risk validation.

Overall, AlgoTrade Pro demonstrates a practical, production-oriented approach to quantitative trading software and reflects end-to-end engineering from requirement analysis to a deployment-ready implementation.

CHAPTER 1: INTRODUCTION

1.1 ORGANIZATIONAL CONTEXT AND INDUSTRY OVERVIEW

Algorithmic trading has become an essential part of modern financial markets due to the need for speed, consistency, and data-driven execution. In the Indian context, retail participation has expanded rapidly, but many traders still depend on fragmented tools for charting, order placement, and analysis. This creates delays, inconsistency in decisions, and weak risk discipline.

AlgoTrade Pro addresses this gap by offering a unified platform where users can analyze stocks, run strategies, simulate trades, and execute broker-connected workflows from one system. The project is aligned with current FinTech requirements: low-latency data flow, secure authentication, modular architecture, and extensible intelligence pipelines.

1.2 CAPSTONE PROJECT PURPOSE AND OBJECTIVES

The purpose of this capstone project is to engineer a practical and scalable trading system that reflects real-world constraints rather than only academic prototypes. The core objectives are:

1. To build a robust backend API layer using FastAPI with asynchronous processing.
2. To design a responsive frontend dashboard for analysis, watchlists, and trade operations.
3. To integrate broker services for market data and trade execution workflows.
4. To implement paper trading and backtesting for strategy validation before live deployment.
5. To incorporate AI-assisted market interpretation without compromising reliability.
6. To enforce risk controls and secure credential management throughout the system.

1.3 SCOPE

The scope of this capstone project includes full-stack development, algorithmic signal workflows, database-backed trade lifecycle management, and real-time data streaming. The implemented system covers:

1. User authentication and protected API access.
2. Trading and watchlist CRUD operations.
3. Live and simulated trading support.
4. Technical and AI-driven analysis modules.
5. Risk management and audit-friendly architecture.

The project focuses on equity workflows for Indian markets and is structured to support future expansion into derivatives and additional broker connectors.

CHAPTER 2: CAPSTONE IMPLEMENTATION AND CONTRIBUTIONS

2.1 ASSIGNED ROLE, RESPONSIBILITIES, AND EXECUTION WORKFLOW

During this capstone, I worked in a full-stack engineering role and handled both product-facing features and core infrastructure modules. Day-to-day responsibilities included API design, React component integration, database modeling, debugging asynchronous flows, and validating end-to-end behavior across backend and frontend layers.

Major responsibilities covered:

1. Building and refining backend services in ``backend/app/services`` for analysis, risk, broker operations, and streaming.
2. Implementing and connecting frontend dashboards in ``components`` with typed service calls from ``services``.
3. Managing data contracts between camelCase frontend payloads and snake_case backend schemas.
4. Creating test and verification scripts under ``backend/scripts`` to validate critical workflows.
5. Maintaining secure configuration boundaries through centralized environment settings.

2.2 MAJOR MODULES COMPLETED

The following modules represent the main capstone deliverables:

1. Trade Lifecycle and Paper Trading Engine

Implemented persistent trade creation, update, and closure flows with explicit trade source handling (``PAPER``, ``MANUAL``, ``AUTO``). This enables realistic paper-trading evaluation without mixing strategy type semantics, while also allowing automated entries and exits to be tracked in the same governed lifecycle.

2. Real-Time Streaming Layer

Implemented browser-to-backend WebSocket flow with a singleton manager that bridges Angel One SmartWebSocketV2 ticks to multiple clients, including reconnect handling and subscription management. The same live feed is consumed by automation workflows for continuous position monitoring.

3. Technical Analysis Pipeline

Implemented indicator-driven analysis using pandas-ta, including RSI, MACD, EMA stacks, ADX, Bollinger bands, ATR, and volume signals. Outputs are consolidated into actionable responses used directly by dashboard views.

4. AI-Driven Analysis and Intelligence

Integrated Gemini-based analysis with circuit-breaker behavior for quota exhaustion, ensuring the platform remains responsive under external API constraints.

5. Backtesting Engine

Implemented strategy execution over historical OHLCV data with a realistic commission model for Indian markets and chart-ready output for result interpretation.

6. Risk Management Guardrails

Implemented pre-trade validations such as maximum order value, daily loss cap, maximum open positions, concentration constraints, and kill-switch support.

The implementation also includes configurable algorithmic execution features such as start/stop control, background watchlist scanning, session restore for open strategy trades, and manual emergency exit handling. These automation behaviors are intentionally integrated with trading, streaming, and risk modules rather than treated as a separate subsystem.

2.3 TECHNICAL ENVIRONMENT AND TOOLSET

The verified stack used in this capstone includes:

1. **Frontend:** React, TypeScript, Vite, Tailwind-based styling.
 2. **Backend:** FastAPI, Python, Uvicorn, async service design.
 3. **Database:** PostgreSQL with SQLAlchemy async ORM.
 4. **Broker Integrations:** Angel One SmartAPI and Zerodha connector support.
 5. **Analytics and AI:** pandas-ta, backtesting.py, Gemini integration.
 6. **Security:** JWT-based auth, bcrypt hashing, encrypted credential vault.
-

CHAPTER 3: KEY LEARNINGS AND SKILL DEVELOPMENT

3.1 TECHNICAL LEARNING OUTCOMES

This capstone strengthened practical understanding of asynchronous architecture in distributed systems. Building a real-time trading application required careful event handling, API boundary design, data model discipline, and robust failure management. Key technical growth areas include:

1. Designing async-first APIs and service orchestration in FastAPI.
2. Handling WebSocket lifecycle and event fan-out to multiple clients.
3. Implementing secure credential handling and token-based authorization.
4. Structuring reusable React components connected to typed services.
5. Balancing AI integration with deterministic fallback behavior.

3.2 PROFESSIONAL AND ENGINEERING PRACTICES

Beyond coding, the project reinforced production engineering habits:

1. Clear separation of concerns across routers, services, models, and schemas.
2. Configuration hygiene through centralized environment management.
3. Defensive programming for external dependencies and rate limits.
4. Validation scripts and health checks to maintain release confidence.
5. Consistent naming and cross-layer data contract discipline.

3.3 PERSONAL AND CAREER REFLECTION

This capstone transformed theoretical understanding into practical capability. Implementing a complete trading platform with security, performance, and reliability constraints provided direct exposure to real software engineering trade-offs. The project significantly improved my readiness for backend-focused and full-stack roles in FinTech and other data-intensive systems.

CHAPTER 4: CHALLENGES FACED AND SOLUTIONS APPLIED

4.1 TECHNICAL AND OPERATIONAL CHALLENGES

The key challenges observed during implementation were:

1. **Symbol-token mismatch across broker APIs** where exchange-ready symbols and broker token IDs must be resolved consistently.
2. **WebSocket fan-out complexity** when one upstream feed serves many frontend subscribers.
3. **Naming mismatch** between frontend camelCase fields and backend snake_case models.
4. **External AI quota limits** that can impact analysis response times.
5. **Schema evolution pressure** while adding new trade attributes without destabilizing existing data.

4.2 SOLUTIONS IMPLEMENTED AND LESSONS LEARNED

The capstone incorporated practical solutions that are directly reflected in the codebase:

1. Implemented a two-step token resolution strategy using instrument data first and API fallback when needed.
2. Used a singleton WebSocket manager with controlled subscription lifecycle and client broadcast handling.
3. Centralized frontend-backend field mapping in service wrappers to prevent payload drift.
4. Introduced circuit-breaker logic around AI calls for graceful degradation during rate-limit events.
5. Used migration and verification scripts to evolve schema and validate data integrity safely.

The most important takeaway was that reliability is primarily driven by architecture choices, not only by algorithm quality.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

AlgoTrade Pro successfully meets the primary goals of this capstone project: a working, modular, and security-conscious algorithmic trading platform grounded in realistic market workflows. The delivered system supports technical analysis, AI-assisted insights, broker connectivity, paper trading, and backtesting in a cohesive product experience.

From an academic perspective, the capstone demonstrates effective application of full-stack engineering principles, asynchronous system design, data modeling, and operational resilience. From an industry perspective, it provides a strong foundation for controlled rollout and future productization.

5.2 FUTURE WORK

Planned extensions for subsequent versions include:

1. Enhanced strategy optimization and parameter search pipelines.
2. Broader broker support and improved account-level analytics.
3. Portfolio-level risk overlays and intraday monitoring dashboards.
4. Scalable task orchestration for long-running research jobs.
5. Expanded AI explainability reports for audit and user trust.

REFERENCES (SUGGESTED)

1. FastAPI Documentation. <https://fastapi.tiangolo.com/>
2. SQLAlchemy 2.0 Documentation. <https://docs.sqlalchemy.org/>
3. PostgreSQL Documentation. <https://www.postgresql.org/docs/>
4. React Documentation. <https://react.dev/>
5. Vite Documentation. <https://vitejs.dev/>
6. Angel One SmartAPI Documentation.
7. Zerodha Kite Connect Documentation.
8. Backtesting.py Documentation. <https://kernc.github.io/backtesting.py/>
9. pandas-ta Documentation.

APPENDIX A: IMPLEMENTATION SCREENSHOTS

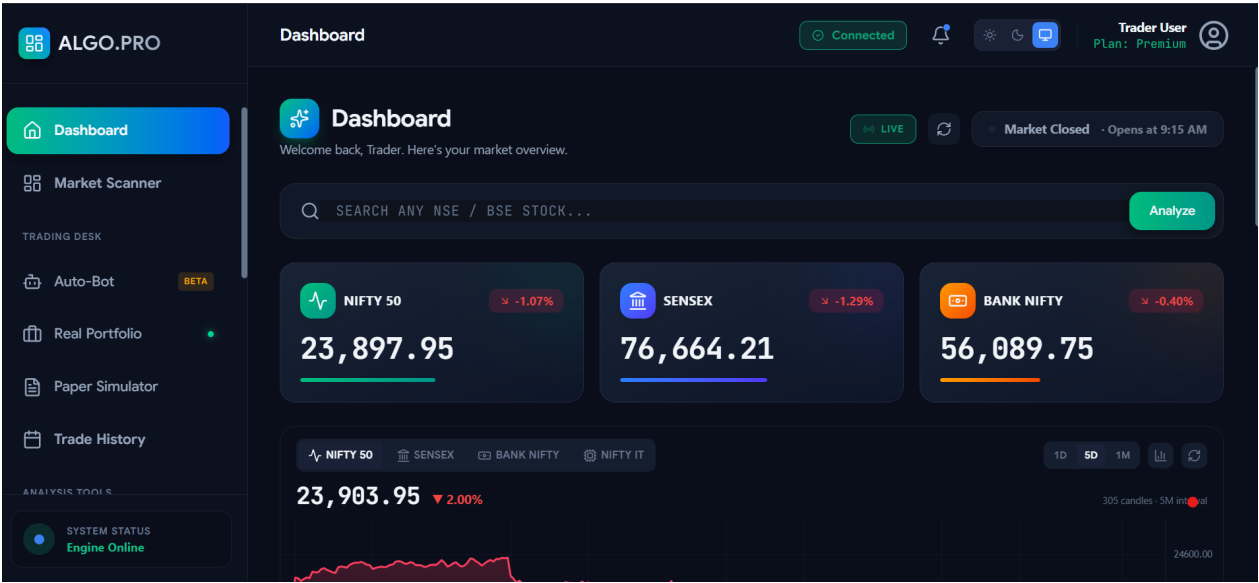


Figure A.1: Dashboard Overview

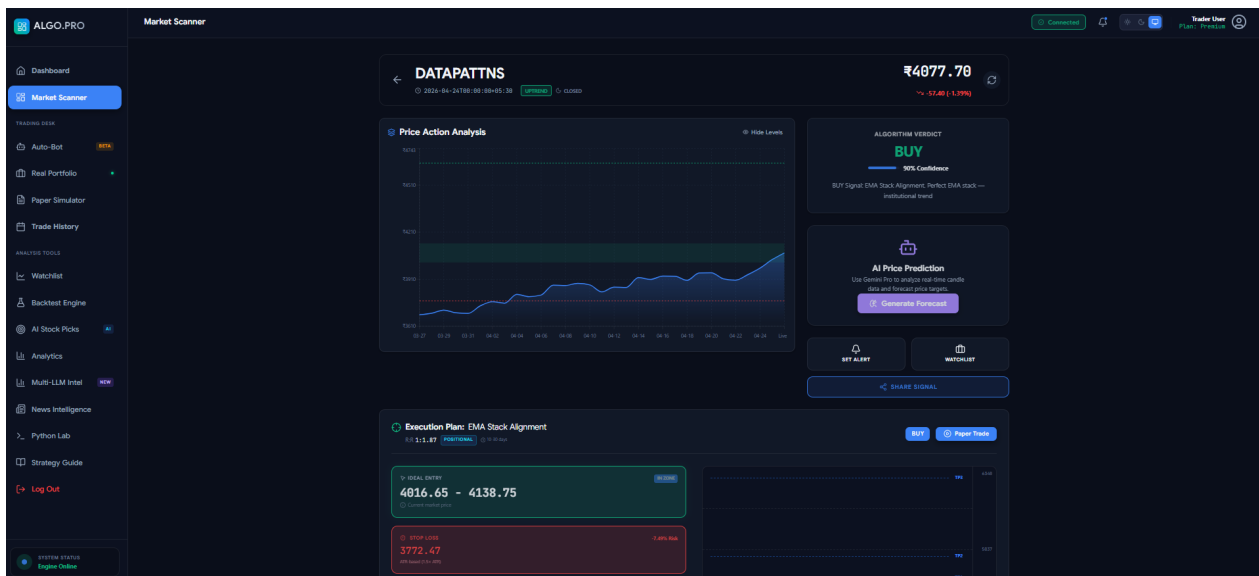


Figure A.2: Market Scanner with Algorithm Verdict and Execution Plan

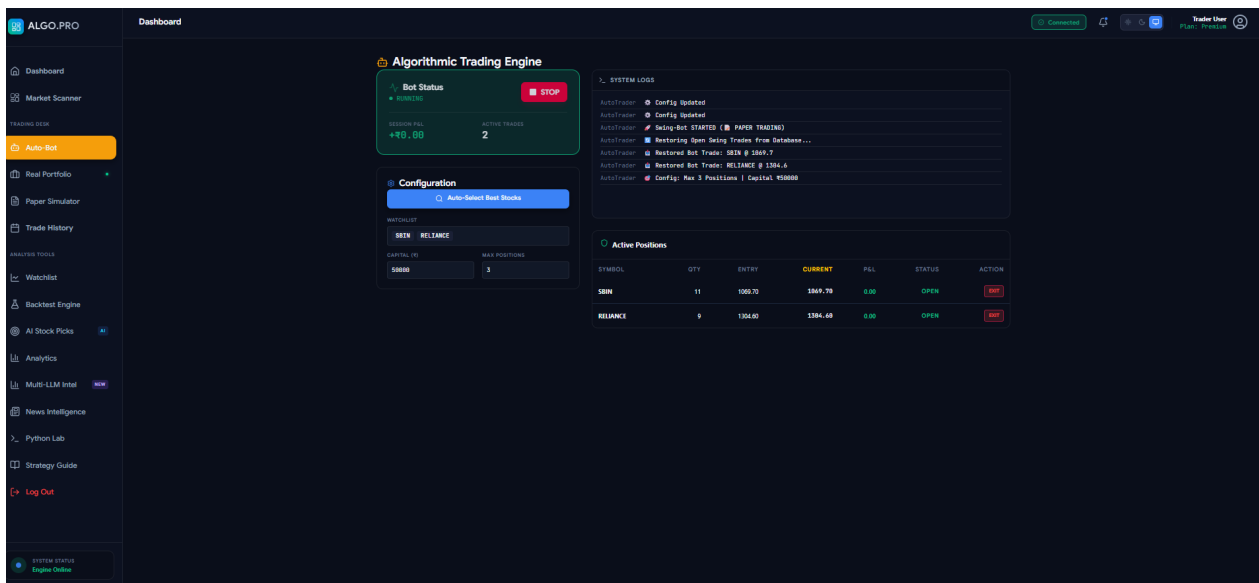


Figure A.3: Algorithmic Trading Engine (Auto-Bot) Configuration and Active Positions

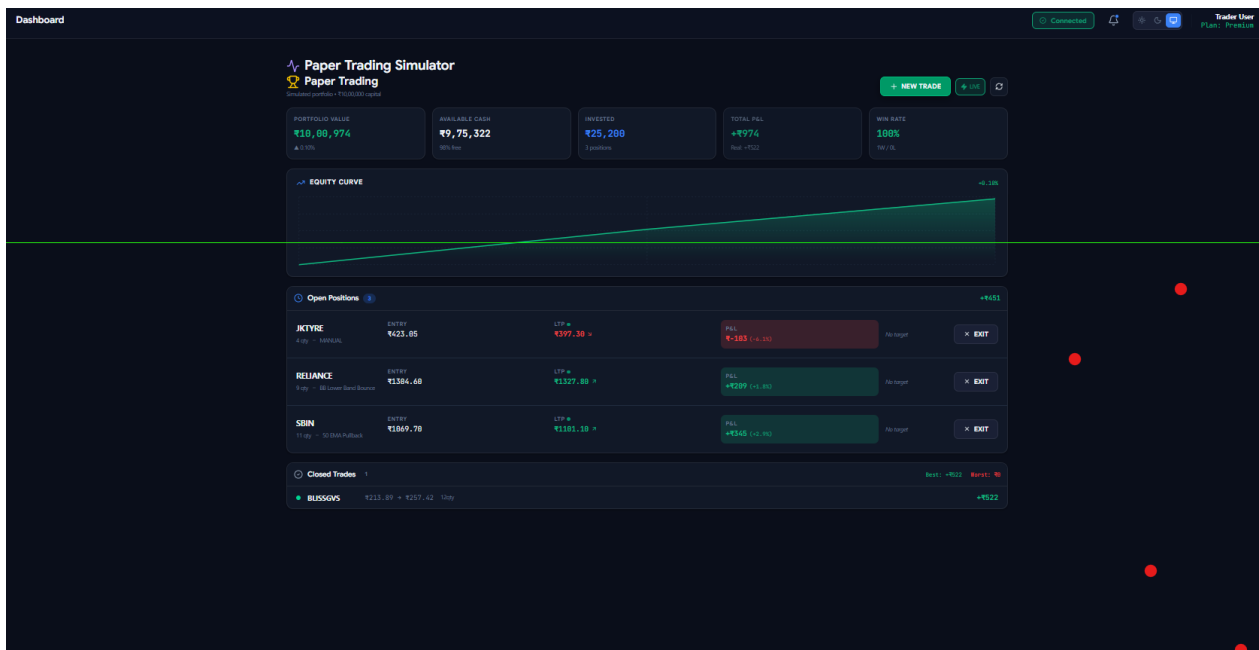


Figure A.4: Paper Trading Simulator with Portfolio Metrics and Open Positions